Gensyn - Delphi Dynamic Parimutuel Markets

Security Assessment

April 2, 2026

Prepared for:
Harry Grieve
Gensyn

Prepared by: **Bo Henderson**

Table of Contents

Table of Contents 1
Project Summary 3
Executive Summary 4
Project Goals 7
Project Targets 8
Project Coverage 9
Codebase Maturity Evolution 11
- Arithmetic 11
- Authentication and Access Controls 11
- Complexity Management 11
- Documentation 12
- Testing and Verification 12
- Transaction Ordering 12

Summary of Findings 13
Detailed Findings 14
- 1. Gateway initialization can be front-run to install a malicious factory 14
- 2. Market creation fails with strict ERC20s and zero market creation fees 16
- 3. Transient reentrancy guard disables market on non-Prague EVMs 17

A. Vulnerability Categories 19
B. Code Maturity Categories 21
C. Code Quality Recommendations 23
D. Design Decision Discussions 26
- Creator Shares and Redemption at Settlement 26
- Liquidation Formula and Order-Independence at Expiry 27
- Configurable Fee Parameters: Risks and Bounds 27
- Validation Layer Redundancy: Buy/Sell Checks Across Gateway, Market, and Library 28
- MIN_TOKENS_DELTA: Tradeoffs of Removal 29

E. Pull Request Reviews 30
- PR #3965: Gateway and Market Refactor 30
- PR #3986: Order-Independent Redemption Model 30
- PR #3997: Faucet Deployer Refactor 31
- PR #4000: Trading Fee Bounds Narrowing 31
- PR #4004: Market Creation Share Liquidation 31

PR #4015: Gateway Initialization Access Control 32
PR #4017: Min-Delta Constants Centralization 33
PR #4025: NatSpec Documentation 33
PR #4031: Error Interface Segregation 34
PR #4035: Gateway-Level Token Storage 34
PR #4044: Delta Enforcement Tests 35
PR #4045: Invariant Test Fixes 35
PR #4049: Foundry Lock Cleanup 35
PR #4055: Post-Settlement Invariant Coverage 36
PR #4075: Fork Test Infrastructure 36
PR #4085: Audit Finding Remediations 37
PR #4102: Fork Test Compilation Fix 37
PR #4126: Code Quality Improvements 38
PR #4144: Finish Scripts and README 38
About Trail of Bits **39**
Notices and Remarks **40**

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Bo Henderson, Consultant
bo.henderson@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
February 17, 2026	Pre-project kickoff call
March 2, 2026	Status update meeting #1
March 10, 2026	Status update meeting #2
March 17, 2026	Delivery of report draft
March 17, 2026	Report readout meeting
April 2, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Gensyn engaged Trail of Bits to review the security of its Delphi Dynamic Parimutuel Markets (DPM).

Delphi DPM is an on-chain prediction market platform where anyone can permissionlessly deploy a new market supporting up to 20 discrete outcomes. Once a market has been created, traders can buy and sell outcome shares, with prices determined by a dynamic parimutuel bonding curve that continuously adjusts based on the relative supply of shares. After trading closes, the market creator submits the winning outcome, collects a share of the accumulated trading fees and a pro-rata payout from the pool based on their seed shares, and winners redeem their remaining shares pro-rata from the pool; if no winner is submitted before a settlement deadline, the market expires, and participants can liquidate their shares for a proportional refund.

A team of one consultant conducted the review from February 23 to March 13, 2026, for a total of three engineer-weeks of effort. Our testing efforts focused on the smart contract codebase in an iterative, advisory capacity; we reviewed the base commit, guided Gensyn through a series of structured design discussions, and assessed pull requests to ensure that each change correctly implemented the intended behavior without introducing new vulnerabilities. With full access to the source code and documentation, we performed static and dynamic testing of the target codebase and pull requests using both automated and manual processes.

Observations and Impact

Trail of Bits reviewed the Delphi DPM smart contracts over a period of active development, engaging not only with a fixed baseline but as a continuous collaborator on incoming pull requests. This model allowed us to intercept issues before they reached the codebase rather than report them after the fact.

The only significant finding in the baseline commit was a low-severity gateway initialization vulnerability (TOB-DELPHIDPM-1) that would have permitted a front-runner to install a malicious factory and bypass market validation. This issue was resolved in PR #4015.

Across a series of PR reviews, two high-severity issues were caught: a missing access control in PR #4004 that allowed anyone to prematurely pull trading fees from market creators, and an initialization check in the same PR that used strict equality against the expected deposit, letting an attacker permanently block all new market deployments with a dust transfer to the next market's address. Both were resolved before merging.

Beyond individual findings, the engagement was oriented toward guiding the protocol's architectural evolution. Design discussions produced analyses that informed concrete code

changes. The two most consequential changes covered order-independent payouts across both settlement paths. One change led directly to a redemption model that drives the pool to exactly zero, enabling stronger invariant testing; the other change led to fee parameter bounds, which establish the validation requirements that any future fee configurability would need to satisfy to avoid permanently locking trader deposits. Additional changes addressed validation layer redundancy, minimum-delta enforcement tradeoffs, and creator-share handling at settlement.

The codebase maturity evolved across multiple quality dimensions over the course of the engagement. The test suite moved from having a key sell-path invariant that silently no-oped and seven others that skipped post-settlement states, to having comprehensive invariant checks applied uniformly across all market lifecycle states. The two settlement paths, which were originally structurally asymmetric and required independent mathematical verification, were ultimately unified through design discussions into a single accounting model whose correctness is now exactly testable. Similar progress was made across documentation, access controls, and other areas, leaving the codebase substantially more auditable, maintainable, and defensible than the one that opened this engagement.

A static analysis run and rapid manual review of the final commit did not surface any new findings.

Recommendations

Based on the codebase maturity evaluation and findings identified during the security review, Trail of Bits recommends that Gensyn take the following steps:

- **No further recommendations.**

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	0
Medium	0
Low	1
Informational	2
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Access Controls	1
Denial of Service	2

Project Goals

The engagement was scoped to provide a security assessment of the Gensyn Delphi DPM protocol. Specifically, we sought to answer the following non-exhaustive list of questions:

- Do pull requests correctly implement their stated intent without introducing regressions in adjacent logic?
- Do architectural refactors, such as moving the token configuration to the gateway or centralizing minimum-delta enforcement, preserve the security properties of the original design?
- Are the proposed design changes to the liquidation formula, fee distribution model, and redemption mechanics mathematically sound and free of economic exploits?
- Do the validation layers and invariant assertions across the gateway, market, factory, and math library form a coherent set of checks with no exploitable gaps?
- Does the bonding curve pricing model correctly compute buy, sell, and liquidation payouts in all market states, and do rounding decisions consistently favor the protocol over traders?
- Are market lifecycle transitions, from open trading through settlement or expiry, correctly gated without any states being entered or bypassed unexpectedly?

Project Targets

The engagement involved reviewing and testing the following targets.

Private Monorepo

Repository	https://github.com/gensyn-ai/node
Start Version	`97b559135aaaf9ff545f7c6de48ebde35c949155`
Final Version	`1ae52605d12d9e3b609d715a44b613ddba652ee4`
Path	`delphi/smart-contracts`
Type	Solidity
Platform	Gensyn

Public Mirror

Repository	https://github.com/gensyn-ai/delphi-smart-contracts
Version	`c400d99f957fee236bb187ba137a80a9004aa9f3`
Type	Solidity
Platform	Gensyn

While the private monorepo was the sole target of this review, a public mirror of the `delphi/smart-contracts/` directory has been provided for transparency. We used the following command to confirm that the `src`, `test`, and `script` directories of the public mirror at the specified commit exactly match those of the private monorepo at the specified path and final commit.

```
find src test script -type f -print0 |\
  LC_ALL=C sort -z |\
  xargs -0 sha256sum |\
  sha256sum
# fb3026cfbba1a936864a137c27360996efa4bdfe55f5aabc3be130f848072187 -
```

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **Static analysis**. We ran three tools against the in-scope contracts and manually triaged all output.
    - Slither produced a batch of high- and medium-severity findings. Manual review revealed that all were false positives stemming from Slither's lack of architectural context.
    - Aderyn surfaced a few code-quality observations.
    - An internal AI-powered vulnerability hunting tool scanned the full attack surface across multiple sessions. We manually consolidated and triaged findings from all runs, identifying several candidate issues that drove subsequent manual review.
- **Dynamic analysis**. We ran the full invariant and unit test suite and used failures and near-failures as entry points for deeper investigation, particularly around edge cases in severely imbalanced markets. In several cases, this analysis generated candidate issues. In others, it built useful intuition about the protocol's economic behavior at extremes and was an important input to the design discussions.
- **Design decision discussions**. We reviewed several areas of the protocol's design where implementation choices involved non-obvious tradeoffs and discussed recommendations with the Gensyn team, iterating toward safe solutions in real time. See appendix D for full documentation of these discussions.
- **PR reviews**. Starting in the second week, we reviewed incoming branches as they arrived, covering changes related to, and sometimes directly resolving, the issues and design discussions mentioned above. See appendix E for the complete list and per-PR findings. The following are highlights from a few notable PRs:
    - PR #4004: The largest PR of the engagement, addressing a substantial refactor of the liquidation path. We identified and promptly eliminated newly introduced risks, including a critical denial of service (DoS) vulnerability.
    - PR #4017: Removed per-market delta floor immutables and centralized enforcement in the gateway. We confirmed that the removed checks were already unreachable due to the `onlyGateway` constraint, so the change eliminates a dead and misleading defense-in-depth measure without weakening the actual security boundary.

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- This engagement focused exclusively on the `delphi/smart-contracts` component. The UI, backend servers, and their interactions with the deployed contracts, including how off-chain systems interpret events, construct transactions, and handle error conditions, were out of scope and not reviewed.

Codebase Maturity Evolution

Trail of Bits reviewed the Delphi DPM codebase under active development, assessing incoming pull requests in parallel with the core audit. The sections below track how each quality dimension changed from the engagement's start to its close.

Arithmetic

Trail of Bits had previously audited an earlier iteration of a closely related codebase with a focus on arithmetic, and the lessons from that engagement carried over: rounding conventions and precision management were correct at the outset of this engagement and remained so. The work here was refinement. Redundant assertions that obscured rather than protected were removed; those guarding genuinely non-trivial invariants were kept. PR #3986 replaced the redemption model with a design that drives the settlement pool to exactly zero; this eliminates stranded dust and making the pool-emptiness invariant exactly testable, which directly enabled the invariant improvements in PR #4055. A design discussion on fee configurability identified a latent arithmetic trap that could permanently lock all trader deposits on a given factory; the team closed it by enforcing a hard ceiling on the fee constant at construction time.

Authentication and Access Controls

The baseline codebase had a gap allowing a malicious actor to hijack gateway initialization, install a factory they control, and bypass all market validation (TOB-DELPHIDPM-1); the fix was delivered in PR #4015. Other issues were caught before merging. PR #4004 introduced a new privileged function with no access restriction, allowing anyone to drain creator fees at any market state; access controls were added. The same PR's initialization check could be exploited with a dust transfer to permanently block new market deployments; the check was relaxed to avoid such DoS vectors. PR #4085 fixed an unconditional token transfer that would cause deployment to revert on certain ERC-20 tokens.

Complexity Management

At the outset, the `SETTLED` and `EXPIRED` market-close paths were structurally asymmetric and the differences were undocumented, requiring independent verification of two mathematical models. Validity checks were scattered across three layers with no clear ownership; several were unreachable, others were redundant, and others obscured where enforcement actually lived.

Through design discussions and PR reviews, the team reached a substantially cleaner architecture. Every check was classified as defense-in-depth, redundant, or a structural gap; the only truly duplicated enforcement was removed in PR #4017, while checks serving as invariant documentation were kept. The settlement asymmetry was resolved so that both paths use the same accounting model. Token configuration was lifted to the gateway level in PR #4035, structurally preventing market–gateway mismatches. Five design-decision

write-ups were produced covering the most complex or sensitive tradeoffs, giving future maintainers durable context for choices that would otherwise be opaque.

Documentation

The codebase arrived with sparse NatSpec, unresolved inline developer questions, and no record of key design decisions. A noteworthy example surfaced during review of PR #3986: a commented-out line in `submitWinner` would, if mistakenly uncommented, have silently overpaid early redeemers at the expense of later ones. PR #4025 added NatSpec throughout all five in-scope contract files. Design-decision writeups document the reasoning behind each major architectural choice.

Testing and Verification

The test suite started out with major structural gaps. The most significant: a key sell-path invariant had silently no-oped since it was written, exercising nothing. Seven other invariants skipped post-settlement market states entirely, and delta-enforcement revert paths had no dedicated unit tests.

PR #4045 corrected the no-op invariant. PR #4055 extended seven invariants to cover all market states, made possible by a redemption model change in PR #4004 that kept protocol accounting valid post-settlement. PR #4044 added targeted unit tests for every delta-enforcement revert path. PR #4075 introduced fork-test infrastructure exercising the protocol against live tokens on a forked network.

Transaction Ordering

Two minor timing issues were present at the start: a newly deployed market proxy was registered before initialization completed, and events were emitted after external interactions rather than before. Both were corrected.

One further issue was caught during the engagement. PR #4085 added reentrancy protection to `initialize`, closing a window where a callback mid-initialization could interact with the not-yet-ready market.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Gateway initialization can be front-run to install a malicious factory	Access Controls	**Low**
2	Market creation fails with strict ERC20s and zero market creation fees	Denial of Service	**Informational**
3	Transient reentrancy guard disables market on non-Prague EVMs	Denial of Service	**Informational**

Detailed Findings

1. Gateway initialization can be front-run to install a malicious factory	
Severity: **Low**	Difficulty: **High**
Type: Access Controls	Finding ID: TOB-DELPHIDPM-1
Target: `src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol`	

Description

The `initialize` function of `DynamicParimutuelGateway` is declared with `external` and `initializer` modifiers; it has no access control and no `_disableInitializers` call in the constructor. The deployment helper `_deployDelphi` of `DelphiDeployer` issues four separate transactions when broadcast via `vm.startBroadcast`: gateway deployment, implementation deployment, factory deployment, and finally `gateway.initialize(legitimateFactory)`. The gap between the gateway deployment transaction and the initialize transaction leaves the gateway open to front-running.

```
function initialize(IDelphiFactory delphiFactory_) external initializer {
    require(address(delphiFactory_) != address(0), "delphiFactory cannot be
address(0)");
    require(address(delphiFactory_).code.length > 0, "delphiFactory must be a smart
contract");
    delphiFactory = delphiFactory_;
}
```

Figure 1.1: Gateway initializer with no caller restriction (`src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol#L33–L40`)

An attacker who calls `initialize` first permanently sets `delphiFactory` to a malicious contract of their choice. The `initializer` modifier prevents any subsequent re-initialization, so the gateway is permanently compromised.

```
function _ifDeployedByFactory(IDynamicParimutuelMarket marketProxy) internal view {
    require(_getInitializedVersion() == 1, "gateway not initialized");
    require(delphiFactory.marketProxyExists(address(marketProxy)), "market proxy not
deployed by factory");
}
```

Figure 1.2: All gateway market operations validate against the stored factory address (`src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol#L482–L486`)

Exploit Scenario

An attacker monitors the mempool and observes the gateway deployment transaction land on-chain. They pre-deploy a malicious `IDelphiFactory` implementation that returns `true` for every `marketProxyExists` call. Before the legitimate `gateway.initialize(legitimateFactory)` transaction is included, the attacker submits (or bundles via MEV) a call to `gateway.initialize(maliciousFactory)`. The legitimate initialize transaction reverts with `InvalidInitialization`. The gateway permanently holds `delphiFactory = maliciousFactory`. The attacker then deploys a malicious market contract whose `buy` function calls `TOKEN.safeTransferFrom(victim, attacker, amount)` instead of crediting shares. When users interact with the attacker's malicious market through the legitimate gateway, the gateway's `_ifDeployedByFactory` check passes and the attacker drains the victims' token allowances.

Recommendations

Short term, add an `address internal immutable DEPLOYER` state variable to `DynamicParimutuelGateway` and set it to `msg.sender` in a `constructor()`. Then add `require(msg.sender == DEPLOYER, "only deployer can initialize")` in the `initialize` function. Because `DEPLOYER` is recorded at construction time and the `initialize` function enforces it, only the deploying account can initialize the gateway, which prevents any front-runner from supplying a malicious factory address.

This issue was fully resolved by PR #4015 in commit `70a215c4`.

2. Market creation fails with strict ERC20s and zero market creation fees

Severity: **Informational**	Difficulty: **High**
Type: Denial of Service	Finding ID: TOB-DELPHIDPM-2
Target: `src/delphi/factory/DelphiFactory.sol`	

Description

The `deployNewMarketProxy` function of `DelphiFactory` calls `safeTransferFrom` to collect the market creation fee unconditionally, without checking whether the fee amount is zero. Some tokens, such as BNB, revert when the transfer amount is zero, blocking market creation.

```
// Interactions: Charge marketCreationFee
TOKEN.safeTransferFrom(msg.sender, MARKET_CREATION_FEE_RECIPIENT,
MARKET_CREATION_FEE);
```

Figure 2.1: Unconditional fee transfer in the `deployNewMarketProxy` function (`src/delphi/factory/DelphiFactory.sol#L67`)

The constructor comment at line 36 explicitly acknowledges that a zero fee is a supported configuration: "Note: marketCreationFee can be zero." `MARKET_CREATION_FEE` is set as an immutable in the constructor and cannot be changed after deployment. `DelphiFactory` is not upgradeable. If the fee is configured as zero against such a token, the result is a complete and permanent denial of service for market creation.

Exploit Scenario

A protocol operator deploys `DelphiFactory` with `marketCreationFee = 0` to allow permissionless market creation without a fee barrier. The configured `TOKEN` is a strict ERC-20 implementation that reverts on zero-value transfer calls. Every subsequent call to `deployNewMarketProxy` reverts at the fee transfer, making it impossible to create any market. This denial of service is permanent, and a full redeployment of the gateway, implementation, and factory is required to restore functionality.

Recommendations

Short term, guard the fee transfer with a zero-value check: only call `safeTransferFrom` when `MARKET_CREATION_FEE` is greater than zero. This prevents the issue because tokens that revert on zero-value transfers are handled without disrupting the normal market creation flow.

This issue was fully resolved by PR #4085 in commit `fd5b1fa8`.

3. Transient reentrancy guard disables market on non-Prague EVMs

Severity: **Informational**	Difficulty: **High**
Type: Denial of Service	Finding ID: TOB-DELPHIDPM-3
Target: `src/delphi/dynamicParimutuel/implementation/DynamicParimutuelMarket.sol`	

Description

`DynamicParimutuelMarket` inherits OpenZeppelin's `ReentrancyGuardTransient`, which implements its lock using `TSTORE` (opcode `0x5D`) and `TLOAD` (opcode `0x5C`) from EIP-1153 (transient storage). The project's `foundry.toml` specifies `evm_version = 'prague'`, so the compiled bytecode emits these opcodes directly with no runtime fallback. OpenZeppelin's documentation explicitly states: "NOTE: This variant only works on networks where EIP-1153 is available."

All five value-flow functions (`buy`, `sell`, `submitWinner`, `redeem`, and `liquidate`) carry the `nonReentrant` modifier.

On any EVM without EIP-1153 support, executing `TLOAD` or `TSTORE` produces an exceptional halt, which consumes all gas and reverts the transaction. Because the `initialize` function does not carry the `nonReentrant` modifier, a market proxy deploys and receives initial liquidity normally. Every subsequent call to the `buy`, `sell`, `submitWinner`, `redeem`, and `liquidate` functions then fails permanently. Because `GATEWAY` is an immutable set at construction time with no upgrade path, there is no recovery mechanism.

EIP-1153 is part of the Prague/Pectra hard fork for Ethereum mainnet (activated April 2025) but is not uniformly available across all EVM-compatible networks. Layer-2 networks and alternative chains may not support it.

Exploit Scenario

The protocol deploys market proxies on an EVM-compatible network that has not yet activated EIP-1153. Market creators provide initial liquidity through the `initialize` function, which succeeds because it carries no `nonReentrant` modifier. When any user attempts to call the `buyExactOut` function of `DynamicParimutuelGateway`, the gateway forwards the call to the `buy` function of `DynamicParimutuelMarket`, which executes the `nonReentrant` modifier. The modifier invokes `TLOAD` to read the reentrancy slot. The EVM encounters an unsupported opcode, halts execution, and reverts the transaction,

consuming all provided gas. Every call to the `buy`, `sell`, `submitWinner`, `redeem`, and `liquidate` functions fails identically. All initial liquidity deposited across every market proxy on the chain is permanently locked with no recovery path.

Recommendations

Short term, add the `nonReentrant` modifier to the `initialize` function of `DynamicParimutuelMarket`. Because `nonReentrant` issues a `TSTORE` on entry, any attempt to create a market on a chain without EIP-1153 support will revert before initial liquidity is deposited, preventing funds from ever being committed to a non-functional market.

Long term, document the minimum EVM capability requirements for safe deployment as part of the protocol's deployment runbook, and add CI checks that verify the target network supports EIP-1153 before any deployment script executes.

This issue was fully resolved by PR #4085 in commit `fd5b1fa8`.

References

- OpenZeppelin ReentrancyGuardTransient
- EIP-1153: Transient Storage Opcodes

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	**Description**
Arithmetic	The proper use of mathematical operations and semantics
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Documentation	The presence of comprehensive and readable codebase documentation
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	**Description**
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Code Quality Recommendations

This appendix contains recommendations for findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Remove dead min-delta checks.** The `buy` and `sell` functions enforce minimum trade sizes directly, duplicating the enforcement that the gateway's quote functions already apply before calling the market. Because `buy` and `sell` are restricted to the gateway via the `onlyGateway` modifier, the market-level checks can never be the first to fire. Even if a bug caused these checks to be reached, the math library's `buyIsValid` and `sellIsValid` functions would still protect the underlying system invariants, running unconditionally further down the call stack. The market-level minimum-delta checks are thus caught between the gateway's user-facing quota checks above and the math library's invariant guards below, contributing nothing at either level. Resolved by PR #4017 in commit `37067d96`.

- **Remove dead fee assertions.** The `grossAmount >= netAmount` assertions in `deductFee` and `addFee` fire immediately before computing `feeAmount` as the difference of those two values, but can never actually trigger: the preceding math guarantees `netAmount` is strictly smaller than `grossAmount` for any non-zero trading fee, and Solidity 0.8.x's built-in underflow protection on the following subtraction covers any remaining edge case. The assertions add no safety value while replacing a semi-informative underflow revert with a less-informative generic panic. Resolved by PR #4126 in commit `7273d08a`.

- **Relax the initialized-version check.** The `ifDeployedByFactory` modifier checks whether the initialized version equals 1 rather than is at least 1. The latter form expresses the actual invariant—the gateway has been initialized at least once—without coupling correctness to a specific version number. The developer's own comment flags this brittleness: if a reinitializer for version 2 were ever added, every market-facing gateway function would permanently revert. The change also aligns the check with its associated error message, which refers to an uninitialized state rather than anything specific to version 1. Resolved by PR #4126 in commit `7273d08a`.

- **Expand the `winningOutcomeIdx` storage comment.** This field serves as both a sentinel and the mechanism that transitions a market to the `SETTLED` status, which gates every value-moving function. This expanded comment should include the following pieces of information.

    - This value is initialized to the maximum uint256; this sentinel value signals that no winning outcome has been set yet.

    - This value can ONLY be set by the creator while the market is `AWAITING_SETTLEMENT`
    - Setting this value will transition the market into the `SETTLED` status.

  Resolved by PR #4126 in commit `7273d08a`.

- **Initialize clones before registering them.** The `deployNewMarketProxy` function registers a new clone in the proxy list before calling `initialize` on it, meaning the registered clone briefly has all storage zeroed. Because `_ifDeployedByFactory` checks only that the gateway itself is initialized and that the proxy is present in the registry, it passes for an uninitialized market, leaving all value-moving functions callable against a proxy with zeroed storage. In that state, `winningOutcomeIdx` is `0` rather than the maximum `uint256` sentinel, so `marketStatus` returns `SETTLED`. No exploit is presently possible: shares cannot be minted until the market is initialized, so no attacker can redeem shares against such an uninitialized clone. However, ensuring that the proxy is fully initialized before registration eliminates this attack surface and keeps the initialization process safe against future code changes. Resolved by PR #4126 in commit `7273d08a`.

- **Move `nonReentrant` first.** All five trading and settlement functions place the `nonReentrant` modifier last, after `onlyGateway` and `ifStatus`: `buy`, `sell`, `submitWinner`, `redeem`, and `liquidate`. Neither existing modifier makes an external call, so there is no present risk. But the best practice is to acquire the reentrancy lock before any other logic runs, so that a future modifier that performs an external call cannot be exploited before the lock is set. Resolved by PR #4126 in commit `7273d08a`.

- **Treat events as effects in CEI.** All five value-flow functions—`buy`, `sell`, `submitWinner`, `redeem`, and `liquidate`—emit their events after the final token transfer rather than before it. Contracts with weaker reentrancy protection can be manipulated into emitting events out of order, leading backend systems to misbehave or crash. The consistent use of `nonReentrant` modifiers across all market functions mitigates this risk, but the current ordering is a gap in the second layer of CEI-based protection. Resolved by PR #4126 in commit `7273d08a`.

- **Remove or implement unused error declarations.** `TokensOutNotPositive` and `WinningOutcomeOutOfBounds` are declared in the market errors interface, but never reverted anywhere. The former appears to have been superseded by `ZeroTokensOut` and is no longer needed. The latter appears to have been intended as a replacement for the string error in `_validOutcomeIdx`, but was never wired up. Unused error declarations create false confidence that validation exists where it does not. Resolved by PR #4126 in commit `7273d08a`.

- **Remove the stale rounding comment.** A comment in `sharesPerOutcomeAtMarketCreation` asks: “Which way should we round this?” The question was answered when PR #4004 changed the return statement from an implicit `mulDiv` call to an explicit `mulDivDown` call, but the comment was not removed at that time. The comment implies uncertainty that no longer exists. Resolved by PR #4126 in commit `7273d08a`.

D. Design Decision Discussions

During the engagement, Trail of Bits held a series of structured design discussions with the Delphi team on topics at the boundary between correctness and deliberate tradeoffs. Each discussion examined a specific design choice in depth, weighed the alternatives, and produced a written analysis. This appendix summarizes the five topics covered.

Creator Shares and Redemption at Settlement

We examined two related questions for the `SETTLED` path: how to handle the market creator's locked winning shares at settlement, and how to ensure that `redeem` payouts are order-independent.

Creator shares. In the original design, `submitWinner` burned the creator's winning shares atomically with the payout. Three options were evaluated:

- **Option Null (status quo):** Burn shares during `submitWinner`. Both paths remain internally consistent, but use structurally different math.
- **Option A (never burn):** Skip the burn and adjust the `redeem` denominator to exclude creator shares via subtraction. Initially judged worse than Option Null because it introduces a fixed-value special case to `redeem` without simplifying the `EXPIRED` path.
- **Option B (always burn, snapshot first):** Burn creator shares at the first `liquidate` call, allowing both paths to use simple pro-rata. The cost is a new state variable, a first-liquidator gas spike, and different payout economics that concentrate payouts toward popular outcomes.

The discussion initially leaned toward Option Null as already correct and Option B as carrying regression risk. However, Option A's original weakness was eventually addressed: rather than subtracting a hardcoded `marketCreatorSharesPerOutcome` from the denominator, `redeem` computes `unclaimedShares = totalSupply - balanceOf(address(this), winningOutcomeIdx)` using a live read. As each redeemer transfers their shares to the contract, `unclaimedShares` falls in lockstep with `_market.pool`, making the accounting uniform across all redeemers, including the creator. This refinement eliminated Option A's special case and made it the preferred approach.

Order-independence. The primary motivation for order-independence in `redeem` is front-running resistance: a redeemer who acts first should not receive a materially larger or smaller payout than one who waits. The decremented-pool formula `pool × sharesIn / unclaimedShares` achieves this. A secondary choice remains between a frozen-pool design, which gives every redeemer exactly the same number of tokens per share but

leaves rounding dust permanently stranded, and the decremented-pool design, which concentrates dust on the last redeemer (who receives a negligibly larger payout) but drives `_market.pool` to exactly zero. This tradeoff is economically inconsequential but carries a concrete engineering benefit: the strict `pool == 0` post-settlement invariant allowed PR #4055 to remove the early-exit guards that had caused invariant checks to skip `SETTLED` markets entirely.

Decision: The refined variant of Option A was adopted. Creator shares are not burned in `submitWinner`; the creator reward is computed via `redeemerReward(pool, creatorShares, totalSupply)`, and `market.pool` is decremented accordingly. `redeem` uses the decremented-pool design. These changes were implemented in PR #3986.

Liquidation Formula and Order-Independence at Expiry

We examined how to compute fair, order-independent payouts for the `EXPIRED` path and how to handle the market creator's shares.

Several pre-engagement commits progressively corrected bugs in the original liquidation formula, resulting in a frozen-pool design that was correct but structurally complex by the time design discussions began. The engagement discussions focused on a cleaner approach: expressing the per-outcome payout using `k` and `sqrt(sumTerm36)` directly as `k × Σ(sharesIn_i × supply_i) / (sqrt(sumTerm36) × tokenDecimalScaler)`. Because `k` and `sumTerm36` are fixed at market close (no mints or burns occur in the `EXPIRED` phase), this formula is inherently order-independent, and the frozen-pool reference is no longer needed. As with the `SETTLED` path, order-independence here is about front-running resistance: a liquidator who acts first should not receive a materially larger payout than one who waits.

Creator shares required separate handling. The approach adopted was a dedicated `_liquidateMarketCreationShares` function triggered automatically on the first `liquidate` call, which pays the protocol and decrements the token pool before any user rewards are computed. Subsequent calls proceed directly to user payouts.

Decision: These changes were implemented in PR #4004.

Configurable Fee Parameters: Risks and Bounds

We analyzed what would happen if the two fee constants, `TRADING_FEES_RECIPIENT_PCT` and `MARKET_CREATION_FEE`, were made configurable by an admin.

For `TRADING_FEES_RECIPIENT_PCT`, a value above `1e18` (100%) causes a deterministic underflow in `submitWinner`: the expression `tradingFees - tradingFeesRecipientCut` reverts unconditionally for every market on that factory,

permanently locking all trader deposits, accumulated fees, and creator liquidity. Any configurable setter must enforce `require(newPct <= 1e18)`.

For `MARKET_CREATION_FEE`, the risk is a front-running exploit: a malicious admin could observe a creator's `approve` transaction and raise the fee before `deployNewMarketProxy` executes, extracting up to the creator's full token balance if the approval was unlimited. Exact approvals provide atomic protection, since the transaction reverts if the raised fee exhausts the allowance before the second pull, but the dominant DeFi pattern of unlimited approvals eliminates this protection. Mitigations for a mutable fee include a hard upper bound and a time-locked setter.

Decision: Both constants remain immutable after deployment, eliminating the admin-inflation and front-running risks entirely. The team promoted `TRADING_FEES_RECIPIENT_PCT` from a hardcoded constant to a per-deployment immutable set in the implementation constructor, giving operators per-factory flexibility within a validated range. The constructor enforces `MAX_TRADING_FEES_RECIPIENT_PCT = 1e18` (100%), which closes the underflow scenario identified above. `MARKET_CREATION_FEE` is an immutable set in the factory constructor, with `MIN_MARKET_CREATION_FEE = 0` and `MAX_MARKET_CREATION_FEE = 100e18` (scaled to the token's decimals) enforced at construction. These changes were introduced in PR #4004. The trading fee range was separately narrowed to 0.5%–5% in PR #4000.

Validation Layer Redundancy: Buy/Sell Checks Across Gateway, Market, and Library

We mapped every input validation check across the three layers (gateway, market, and math library) to determine which checks provide defense-in-depth, which are genuinely redundant, and which occupy structural gaps.

The math library contains several guards that are unreachable given well-formed gateway input: `ZeroTokensIn`, `ZeroSharesOut`, and `BuyTooSmall` in `buyIsValid`, and `ZeroSharesIn` and `ZeroTokensOut` in `sellIsValid`. The gateway similarly contains a `ZeroNetTokensIn` check immediately superseded by the stricter min-delta check. The `ifStatus(OPEN)` modifiers on `buy` and `sell` are unreachable via the gateway, since the gateway pre-checks market status before dispatching. None of these conditions can be triggered by any path through the current system.

The genuinely duplicated checks were identified separately: the market-level min-delta `require` statements in `buy` and `sell` duplicated checks already enforced in the gateway's quote functions, with no additional documentation or defensive value, since any caller that bypassed the gateway would also bypass these market-layer copies.

One structural gap was identified: the `OutcomeSupplyBelowMinDelta` check, which prevents a sell from leaving unsellable dust supply, existed only in the gateway view layer.

The check requires knowledge of `MIN_SHARES_DELTA`, a gateway-level constant, making the math library an inappropriate home for it; the gateway is the correct enforcement point.

Decision: The market-level min-delta `require` statements were removed as part of PR #4017, eliminating the only truly duplicated enforcement. The remaining checks, including the unreachable library guards, `ZeroNetTokensIn`, and `ifStatus(OPEN)` on `buy` and `sell`, were retained by design. Each serves as machine-readable documentation of a mathematical or lifecycle invariant that must hold for the system to be sound, and each provides a defensive backstop if the upstream gateway is ever modified or bypassed. The `OutcomeSupplyBelowMinDelta` check was added to the gateway's `quoteSellExactIn` in PR #3965, correctly placed at the layer that owns `MIN_SHARES_DELTA`.

`MIN_TOKENS_DELTA`: Tradeoffs of Removal

We assessed the consequences of removing all `MIN_TOKENS_DELTA` checks from the gateway and market contracts.

No safety invariant depends on `MIN_TOKENS_DELTA`. The math library enforces independent guards (`ZeroTokensIn`/`ZeroTokensOut`, `BuyTooSmall`/`SellTooSmall`, `SqrtOverlap`, price validity, and pool solvency) that prevent fund loss, free share minting, and fee theft regardless of whether the minimum is present. `MIN_SHARES_DELTA` also remains enforced.

Removing `MIN_TOKENS_DELTA` has three consequences, none of which is a security vulnerability: dust trades at mathematically valid prices are permitted; spam protection weakens in cheap-share markets where `MIN_SHARES_DELTA` alone costs near-zero tokens; and a rounding edge case in `sell` produces a parameterless `ZeroTokensOut` error rather than a diagnostic `TokensOutBelowMin` error. The checks are worth retaining for spam-prevention and debuggability, but their removal does not introduce any exploitable condition.

Decision: The checks were retained in the gateway layer, and the redundant market-layer copies were removed. Before the fix, `MIN_TOKENS_DELTA` and `MIN_SHARES_DELTA` were per-market immutables in the market contract and in the gateway's quote functions. Because `buy` and `sell` are guarded by `onlyGateway`, and the gateway always runs the quote functions before executing a trade, the market-level `require` statements were unreachable in practice. PR #4017 removed the per-market immutables and their corresponding `require` statements, centralizing `MIN_TOKENS_DELTA` as a gateway immutable computed from `_MIN_TOKENS_DELTA_18 / TOKEN_DECIMAL_SCALER`. The economic guardrails and diagnostic error messages are preserved; the dead code is eliminated.

E. Pull Request Reviews

During this engagement, we reviewed a series of pull requests submitted by the Gensyn team against the code in the `delphi/smart-contracts` subdirectory. Reviews are listed below in pull-request order.

PR #3965: Gateway and Market Refactor

PR #3965 was merged via squash commit `3b935a0f`.

Three commits refactoring the gateway and market contracts. Changes included extracting `sqrt` values into named local variables in the quote functions, introducing an `ifOpen` modifier to replace duplicated inline status checks, simplifying the minimum-delta check to use the constant directly, adding an `OutcomeSupplyBelowMinDelta` guard that prevents sells leaving outcome supply in a dusty non-zero zone below the minimum, and improving the checks-effects-interactions ordering in `_liquidateMarketCreationShares`. Several dead commented-out code blocks were removed, and typos were corrected.

We noted that the `assert(!marketCreationSharesLiquidated)` inside the rewritten `_liquidateMarketCreationShares` is unreachable: both call sites guard with `if (!marketCreationSharesLiquidated)` or `require(!marketCreationSharesLiquidated)` before entering the function. Because the assertion reads a warm storage slot, its incremental gas cost is negligible; keeping it as an additional safety layer is reasonable. We also questioned whether the new `marketCreationSharesLiquidated` and `marketCreationSharesValue` gateway wrappers were necessary, since both are side-effect-free views that callers could query directly on the market; the team kept them for consistency with the rest of the gateway's interface.

PR #3986: Order-Independent Redemption Model

PR #3986 was merged via squash commit `23426503`.

This branch replaces the frozen-pool redemption model with a decremented-pool design. Each user receives exactly $P \times U_i / S$ regardless of redemption order, where P is the post-settlement pool, S is the total supply, and U_i is the user's share count. The last redeemer absorbs accumulated round-down dust, driving `_market.pool` to exactly zero with no stranded balance.

We raised several code quality issues, all of which were addressed before merging. A stale comment asserting "Do not decrease market.pool, to keep redemptions order-independent" directly preceded the new `_market.pool -= tokensOut` decrement and was removed. The commented-out `_burn` call and unresolved developer questions in the `submitWinner` function were also removed; had that line been accidentally uncommented, it would have zeroed `balanceOf(address(this))`, corrupted the

`unclaimedShares` denominator, and silently overpaid early redeemers at the expense of later ones. On the test side, the pool-emptiness assertion was reinstated as an exact `assertEqDecimal` check rather than an approximate one, and `invariant_AssetsAreCorrect` was updated to enforce the balance invariant during the `SETTLED` phase rather than returning early. The design itself was praised for sweeping dust away through accounting that makes the pool-emptiness invariant exactly testable rather than merely approximate.

PR #3997: Faucet Deployer Refactor

PR #3997 was merged via squash commit `fdd4e6ee`.

This PR refactored `GensynFaucetDeployer` from accepting an `Addresses` struct to flat parameters, added an optional implementation-reuse path with a `TokenMismatch` guard, and added a standalone `DeployFaucet.s.sol` script with a Makefile target and README documentation. Typos in the README were also corrected.

Two minor code quality items were acknowledged during review and deferred from this PR: an unused `GENSYN_MULTSIG` declaration in `GensynFaucetUpgrades.t.sol` (later removed in PR #4044), and an inconsistency between the new `TokenMismatch` custom error and the five string-`require` calls in `_validateGensynFaucetArgs` (later replaced with typed custom errors in PR #4044).

PR #4000: Trading Fee Bounds Narrowing

PR #4000 was merged via squash commit `10577efd`.

A two-line change raising `MIN_TRADING_FEE` from 0% to 0.5% and lowering `MAX_TRADING_FEE` from 10% to 5%. The math is correct at both new bounds. The `if (tradingFees > 0)` guard in `liquidate` remains load-bearing: it prevents zero-value token transfers when a second liquidator calls in after fees have already been swept, which matters for ERC-20 tokens that revert on zero transfers. An inline comment had described it as a gas-inconsistency guard, which was stale after the new non-zero minimum; correcting the comment was left as a follow-up. The `TradingFeeTooLow` revert path was previously unreachable (a `uint256` is never below zero), but with the new non-zero minimum, it became live. We recommended adding a mirror test for the newly reachable path, and the PR delivered it, matching the structure of the existing `TradingFeeTooHigh` coverage.

PR #4004: Market Creation Share Liquidation

PR #4004 was merged via squash commit `eef1f74b`.

This branch introduced the two most structurally significant changes of the engagement: a standalone market-creation-share liquidation path and a transfer-to-self redemption model.

We identified two problems in this branch, both of which were resolved before it was merged.

The first was a missing access control on `liquidateMarketCreationShares`. In its initial form, the function was `public` with no caller restriction and no lifecycle guard, allowing anyone to drain trading fees from market creators or the liquidity pool at any market state. The team added `onlyGateway` and `ifStatus(EXPIRED)` modifiers, resolving the issue. Two lower-priority items were noted and explicitly deferred rather than blocking the merge: the standalone `liquidateMarketCreationShares` gateway path emits no event, which limits off-chain indexers' ability to observe creator-share liquidations; and `assert` was used in place of named `revert` for the constructive invariants in `initialize`, producing an opaque `Panic(0x01)` if triggered rather than a descriptive error.

The second was a donation-DoS vulnerability in `initialize`. The balance check used strict equality (`!=`) against `initialLiquidity_`. Because market proxies are deployed via `CREATE`, an attacker could predict the next proxy address and front-run its deployment with a small token transfer, causing the balance check to revert the initialization permanently. At negligible cost, any token holder could block all new market deployments indefinitely. The team changed the check from `!=` to `<` and swept any excess balance (including donated dust) to `TRADING_FEES_RECIPIENT`, resolving the issue.

The transfer-to-self redemption model replaced the previous burn-on-settlement design. Rather than burning redeemed shares, the new code transfers them to `address(this)`, preserving `totalSupply` and keeping spot prices frozen post-settlement. The pro-rata payout formula is mathematically sound and order-independent.

PR #4015: Gateway Initialization Access Control

PR #4015 was merged via squash commit `70a215c4`.

This branch added a `DEPLOYER` immutable to `DynamicParimutuelGateway` and a matching `require(msg.sender == DEPLOYER)` guard in `initialize`, resolving TOB-DELPHIDPM-1. The fix correctly closes the multi-transaction initialization window that would have allowed a frontrunner to supply a malicious factory address and bypass all market validation. The branch also wired in the previously stubbed-out `_verifyDelphiArgs` call in `DelphiDeployer`, adding zero-address guards on the deployment configuration arguments.

Three of the four unit tests requested in review were delivered; one gap remains. The success-path test (confirming the `delphiFactory` getter is readable after initialization), the double-initialize revert, and the non-deployer revert were all added. A test verifying that a non-contract (EOA) factory address is rejected was left for a future pass. Review comments also asked for the `Initializable` dependency to be dropped in favor of a plain `require(address(delphiFactory) == address(0))` guard, which better

reflects the gateway's non-proxy lifecycle; this refactor was acknowledged as a low-priority change that would not alter observable behavior, and was similarly deferred.

PR #4017: Min-Delta Constants Centralization

PR #4017 was merged via squash commit `37067d96`.

This branch removed `MIN_SHARES_DELTA` and `MIN_TOKENS_DELTA` as per-market immutables and centralized them in the gateway. The apparent removal of bounds checks from `market.buy` and `market.sell` is safe: identical checks already existed in `quoteBuyExactOut` and `quoteSellExactIn`, and since `buy` and `sell` are `onlyGateway`, no input can reach the market execution path without first passing the gateway's checks. A `minTokensDelta(marketProxy)` view was added to the gateway to compute the token-decimal-adjusted minimum.

The PR also introduced a new `test_Gateway_MinTokensDelta` unit test that verifies the arithmetic of `minTokensDelta`'s return value. We flagged that the `TokensInBelowMin` and `TokensOutBelowMin` revert paths, covered probabilistically by invariant tests, would benefit from dedicated unit tests for speed and determinism; the team acknowledged this and delivered fixes for both in PR #4044. We also flagged the two state variables, `gateway` and `dynamicParimutuelGateway`, that the refactored test setup left pointing to the same object; this duplicate was likewise cleaned up in PR #4044.

PR #4025: NatSpec Documentation

PR #4025 was merged via squash commit `c9f2b8e3`.

This branch adds NatSpec documentation throughout all five in-scope contract files—`DynamicParimutuelGateway`, `IDynamicParimutuelGateway`, `DynamicParimutuelMarket`, `IDynamicParimutuelMarket`, and `DynamicParimutuelMath`—with no changes to production logic. Contract-level `@title`, `@notice`, and `@dev` tags are added to each file; events, constants, immutables, modifiers, constructors, and all public, external, and internal functions receive structured parameter and return documentation. Implementation functions that satisfy interface declarations use `@inheritdoc` rather than duplicating the interface text.

No security issues were identified. Over three batches of commits, all review recommendations were resolved: the redundant inline comments inside `initialize` were removed, the simple view functions in `IDynamicParimutuelMarket` were converted from `@notice`-plus-`@return` to a single descriptive `@return` tag, and the duplicate derivation block comments in `DynamicParimutuelMath` were removed.

PR #4031: Error Interface Segregation

PR #4031 was merged via squash commit `e45a98dd`.

This branch is a pure structural refactoring: error definitions are extracted from the main contract interfaces (`IDynamicParimutuelMarket`, `IDynamicParimutuelGateway`, `IDelphiFactory`, and `DynamicParimutuelMath`) into dedicated error interfaces, and a protocol-wide `IErrors` aggregator is introduced so integrators have a single import for all protocol error selectors. The original interfaces inherit from their corresponding error interfaces, so no existing code required changes and no ABI surface changed.

Three errors (`ZeroTokensIn`, `ZeroSharesIn`, `SharesInExceedSupply`) that were independently declared in both the math library and a higher-level interface are consolidated into `IDynamicParimutuelMathErrors` and pulled into `IDynamicParimutuelGatewayErrors` and `IDynamicParimutuelMarketErrors` via inheritance. This eliminates textual duplication with no ABI impact: the selectors were identical before and after the refactor. One semantic distinction was noted but deferred to a future PR: `ZeroSharesIn` now lives solely in `IDynamicParimutuelMathErrors` yet is also thrown by the market's `liquidate` function when the liquidator holds zero shares. This is a distinct condition that would be better named `ZeroSharesHeld`.

PR #4035: Gateway-Level Token Storage

PR #4035 was merged via squash commit `8fd39dce`.

This branch moved `TOKEN`, `TOKEN_DECIMAL_SCALER`, and `MIN_TOKENS_DELTA` from per-market-proxy values to gateway-level immutables, then removed the `token` constructor parameter from `DynamicParimutuelMarket` entirely. Before the branch, the gateway had no constructor and retrieved token configuration from each market proxy on every quote call via three view functions. After the first stage, the gateway was constructed with an `IERC20Metadata token_` argument, validates that the token has between six and 18 decimals, and exposes `TOKEN`, `TOKEN_DECIMAL_SCALER`, and `MIN_TOKENS_DELTA` as public immutables. After the second stage, the market constructor reads `TOKEN` and `TOKEN_DECIMAL_SCALER` directly from the gateway rather than accepting them as arguments, eliminating the earlier risk of a market being initialized with a token that differs from its gateway's token.

No issues remained in the branch's final state. The token-consistency concern raised during the review of the first seven commits—that the gateway–factory token pairing was unchecked at `initialize` time—is fully resolved in the final commit. The three components form a consistency triangle: the market reads `TOKEN` from the gateway at construction; the factory reads `TOKEN` from the market implementation at construction; and, because two edges are now guaranteed, the third follows by transitivity.

PR #4044: Delta Enforcement Tests

PR #4044 was merged via squash commit `d04c30c5`.

Six commits covering test hygiene and minor production hardening. The branch removed a redundant `dynamicParimutuelGateway` alias in `Delphi.t.sol`, added five targeted unit tests covering every delta-enforcement revert path in the gateway quote functions (`SharesOutBelowMinDelta`, `SharesInBelowMinDelta`, `OutcomeSupplyBelowMinDelta`, `TokensInBelowMin`, `TokensOutBelowMin`), switched the test token to six decimal places (required for the `TokensIn`/`TokensOut` revert paths to be numerically reachable), removed the unused `GENSYN_MULTSIG` variable, and replaced five string-based `require` calls in `GensynFaucetDeployer` with typed custom errors. One minor inconsistency: the commit that removes magic-number literals from `_setUpMarket` introduces a new literal: the `minSharesDelta * 1e8` multiplier in `test_Gateway_Reverts_TokensOutBelowMin`. Because the new literal is well-commented and appears in only one place, we agreed that extracting it to a named constant would be overkill. The branch as a whole improves test readability by pulling constants directly from the implementation, organizing new unit tests cleanly, and consistently applying custom-error patterns throughout.

PR #4045: Invariant Test Fixes

PR #4045 was merged via squash commit `465c4ad7`.

This branch fixed several broken or inactive invariant tests and incorporated the production source changes from `sc/renan/liquidate`. The most significant fix addressed `invariant_CanAlwaysSellEverything`, which had been silently no-oping since it was written: `outcomesWithShares` was declared but never populated, so the inner loop never executed, and the invariant never checked anything. The fix populates the array via a new `userOutcomesWithShares` view on the handler. Additional fixes removed the unused `RedeemArgs.redeemerCount` field (dead code consuming fuzzer budget), explicitly bounded buyer addresses to a controlled range via `MAX_TRADER_COUNT`, and corrected a false-positive price assertion. The fixes were clean with no new issues introduced, and the PR received positive feedback for restoring meaningful coverage to invariants that were silently no-oping.

PR #4049: Foundry Lock Cleanup

PR #4049 was merged via squash commit `a7f16634`.

This was a purely mechanical housekeeping commit that removed stale path-prefixed duplicate keys from `foundry.lock` and a dead `lib/balancer-v3-monorepo` entry. All three surviving dependencies remained pinned to exactly the same commit hashes as before. No recommendations were raised, and the cleanup was approved as worthwhile housekeeping with no regressions.

PR #4055: Post-Settlement Invariant Coverage

PR #4055 was merged via squash commit `f8c16b9a`.

This was a single-commit, test-only branch that removed early-exit guards from seven invariant functions. The guards caused the invariants to skip `SETTLED` and `EXPIRED` markets. Removing them was enabled by a recent change because `redeem` and `liquidate` both transfer shares to `address(this)` now rather than burning them, leaving `totalSupply` and all price-related quantities unchanged post-settlement. The `invariant_Prices_BelowK` invariant retains its `SETTLED/EXPIRED` guard; we noted that, since the per-outcome price bound is equally valid in those states, the guard could also be removed, though we left that as a minor follow-up. Bringing seven invariants to bear regardless of market status represents an unambiguous improvement to test coverage.

PR #4075: Fork Test Infrastructure

PR #4075 was merged via squash commit `78195d0d`.

This branch introduces fork-test infrastructure for running the Delphi DPM protocol against live Gensyn testnet and mainnet state. The new file `test/fuzz/DelphiForkTest.t.sol` deploys a fresh Delphi system on a forked chain and exercises a fuzzed buy-sell-buy-redeem-or-liquidate lifecycle against the on-chain USDC token. Per-network constants (block numbers and USDC addresses) were initially hardcoded as Solidity constants and subsequently moved to a config file at our request. The CI script is updated to exclude fork tests from automated runs. A `viaIR = true` change introduced in the first commit was reverted before the branch reached its final state.

No issues were identified in the final state of the branch. The `sellExactIn` try-catch correctly catches only `TokensOutBelowMin`: given that `quoteBuyExactOut` passed, selling back the exact same shares guarantees the `sqrt` difference is large enough that `SellOverlap` and `GrossTokensOutNotPositive` cannot fire; only fee asymmetry can push the sell payout below `minTokensDelta`.

We recommended placing per-network configuration in JSON format under the existing `script/input/` directory, consistent with the pattern elsewhere in the codebase, and suggested a `cast block-number` + `jq` one-liner to automate block-number refreshes. Instead, the team moved the values to `config/networks.toml`, which is a reasonable alternative that achieves the same basic goal. Several other review suggestions were incorporated before merging: the test contract was renamed from `GensynToken_Test` to `DelphiFork_Test`, and two inline comments that incorrectly described market-proxy approvals as gateway approvals were corrected. We also flagged a missing comment in `foundry.toml` documenting the `viaIR` consistency requirement: that its value must stay synchronized between test and production builds once contracts are deployed. This comment would help future maintainers, but it was deferred for a subsequent commit.

PR #4085: Audit Finding Remediations

PR #4085 was merged via squash commit `fd5b1fa8`.

This branch makes two targeted fixes to resolve TOB-DELPHIDPM-2 and TOB-DELPHIDPM-3. In `DynamicParimutuelMarket`, the branch replaces `ReentrancyGuardTransient` with the standard `ReentrancyGuard` and adds `nonReentrant` to the `initialize` function. In `DelphiFactory`, it wraps the market creation fee transfer in an `if (MARKET_CREATION_FEE > 0)` guard.

No security issues were identified. Both changes are mechanically correct and fully resolve the reported issues. The `ReentrancyGuard` switch is compatible with the clone-proxy architecture: OpenZeppelin v5 stores the reentrancy flag in an ERC-7201 namespaced slot that begins at zero in each fresh clone, and the guard's `ENTERED` sentinel (2) is distinct from that initial zero, so the lock functions correctly before and after `initialize`. We noted that OpenZeppelin v5.5.0 marks `ReentrancyGuard` as deprecated in favor of `ReentrancyGuardTransient`. Adding `nonReentrant` to `initialize` is a sound defensive measure; without it, a callback from the `TOKEN.safeTransfer` call at the end of `initialize` could successfully invoke a `nonReentrant` gateway function against the newly deployed market proxy, since the market's state is fully set by that point and the factory has already registered the proxy. In practice, the attack requires a hook-aware token or a malicious `TRADING_FEES_RECIPIENT`, neither of which reflects the protocol's expected deployment model, but the protection is correct regardless. The zero-fee guard in `deployNewMarketProxy` is a straightforward correctness fix: some ERC20 tokens revert on zero-value transfers, and because `MIN_MARKET_CREATION_FEE` is zero, a factory deployed with no fee would have unconditionally attempted a zero-value transfer before this fix.

PR #4102: Fork Test Compilation Fix

PR #4102 was merged via squash commit `286452e8`.

This single-commit, test-only branch restores compilability of `test/fuzz/DelphiForkTest.t.sol` after the interface refactoring in `sc/renan/token-gateway`. The prior branches extracted error definitions into `IDynamicParimutuelGatewayErrors` and type definitions (including `MarketConfig`) into `IDynamicParimutuelMarketTypes`, inheriting both from the top-level gateway and market interfaces. Because Solidity does not allow referencing a struct or error defined in a parent interface through the child interface name, the fork test needed to import the defining interfaces directly. The branch adds those two imports, updates the three stale references accordingly, and adds a `forge-lint: disable-next-line(unsafe-cheatcode)` suppression before `vm.readFile` to acknowledge the intentional filesystem access. No production contract logic has been changed, and no issues were identified.

PR #4126: Code Quality Improvements

PR #4126 was merged via squash commit `7273d08a`.

This branch resolves all remaining Code Quality Recommendations that had not been addressed by earlier pull requests.

One problem was resolved before the branch reached its final state. The initial commit applied the event-before-interaction pattern incorrectly to the `redeem` and `liquidate` functions of `DynamicParimutuelGateway`. Both functions return their output through named return variables that are zero-initialized at function entry; placing the event emission before the market call caused `GatewayRedemption` and `GatewayLiquidation` to fire with all-zero values regardless of what was actually redeemed or liquidated. A subsequent commit restored the correct ordering (market call first, emit after), so the events carry actual amounts. The final state is correct.

One minor copy-paste error survived to the final state: the comment `// Interactions: Pull tokens to seller` inside the `buy` function of `DynamicParimutuelMarket` should read "from buyer." We noted this issue during review, and given the scope of changes already in the branch, agreed it should be addressed in a follow-up commit.

PR #4144: Finish Scripts and README

PR #4144 was merged via squash commit `1ae52605`.

This branch is a scripts and documentation cleanup with no production contract changes. It fixes a stale `Makefile` target introduced by a prior PR, but the more substantive addition is `DeploySafe.s.sol`, a new deployment script for provisioning a Gnosis Safe multisig proxy. The script reads singleton address, proxy factory address, owners, and threshold from a JSON configuration file, applies structural validation, and deploys the proxy. A `deploy-safe` Makefile target is wired in with the same network, broadcast, and verification flag discipline as the other deployment targets. The `README.md` is expanded with a new section covering Safe deployment and operations.

We identified one code quality recommendation: the `README.md` section describing Safe deployment instructs users to fill out `DeploySafe.json` but provides no field-level documentation, unlike the analogous faucet section which includes a full parameter table. This recommendation was acknowledged as non-blocking and deferred.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report to be business confidential information; it is licensed to Gensyn under the terms of the project statement of work and is intended solely for internal use by Gensyn. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the Trail of Bits Publications page. Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit its use.